Le Mans Université

Licence Informatique *2ème année*

Module 174UP02 Rapport de Projet

# JurassicWar

Solène Orieux, Romane Saint-Léger, Hannah Sergent

12 avril 2026

# Table des matières


**1 Introduction** **3**
1.1 Contexte . . . 3
1.2 Énoncé du problème . . . 3
1.3 Objectifs initiaux du projet . . . 3

**2 Conception** **4**
2.1 Principales règles du jeu . . . 4
2.2 Foncionnalités prévues . . . 5

**3 Organisation du travail** **6**
3.1 Répartition des tâches . . . 6
3.2 Planning prévisionnel . . . 6
3.3 Outils de programmation utilisés . . . 7

**4 Developpement** **8**
4.1 Représentation du terrain par une matrice . . . 8
4.2 Positionnement et mobilité des dinosaures . . . 8
4.3 Système d'actions et outils à disposition du joueur . . . 11
4.3.1 Mécanique des changements de tours . . . 11
4.3.2 Infliger un coup mortel : bombe . . . 12
4.3.3 Blesser un dinosaure : armes de tir . . . 14
4.3.4 Autre méthode de déplacement : grappin . . . 16
4.4 Interface graphique et utilisateur . . . 17
4.4.1 Utilisation du module SDL . . . 17
4.4.2 Création d'un menu principal . . . 17
4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur . . . 18

**5 Résultats et Conclusion** **20**

**6 Bibliographie** **21**

**7 Annexes** **22**
7.1 Jeux de tests . . . 22
7.2 Traces de débogage . . . 27
7.2.1 Correction du système de collisions de la bombe . . . 27
7.2.2 Correction du point d'attache du Grappin . . . 30
7.2.3 Correction des gestions de zones pour le placement des dinosaures . . . 33

# 1 Introduction

Cette introduction présente le travail collaboratif que nous avons mené pour créer un jeu vidéo.

## 1.1 Contexte

Nous avons décidé de programmer un projet de jeu que nous avons nommé Jurassic War. Il est réalisé dans le cadre de la formation de L2 informatique de l'université du Mans pendant la période de janvier à avril 2026. Ce projet a été développé en langage C avec la librairie SDL 2. Ce jeu propose de déplacer des dinosaures avec différents moyens, d'utiliser des armes afin que deux équipes s'affrontent et s'éliminent.

## 1.2 Énoncé du problème

Au travers de ce rapport, nous allons décrire les étapes qui nous ont permis de mener à bien ce projet et de décrire les outils utilisés.
Dans une première partie, nous allons exposer nos choix de conception, avec une description des principales règles du jeu, et de ses fonctionnalités prévues. La deuxième partie, détaille l'organisation de notre travail, dont la répartition des tâches, le planning prévisionnel et les outils de programmation utilisés. Ensuite, nous abordons le développement technique du jeu. Dans cette section, nous détaillons la modélisation du terrain par matrice, les algorithmes de mobilité et de positionnement des entités, et l'implémentation du système d'actions, armes, physique, interface SDL. Enfin, nous présentons les résultats obtenus et notre bilan en conclusion. Une annexe complète ce rapport, vous permettant de prendre connaissances des tests et débogages effectués. Vous pourrez retrouver les fichiers sources sur notre dépôt Git.

## 1.3 Objectifs initiaux du projet

L'objectif principal de ce projet est de concevoir un jeu de stratégie au tour par tour, intégrant des mécaniques de jeu variées et une physique réaliste. Nos priorités sont les suivantes :

- **Système de jeu :** Implémenter un système de combat comprenant quatre armes de tir, une bombe, un grappin pour la mobilité et une potion de soin.
- **Dynamique de jeu :** Gérer l'alternance des tours, déclenchée par la fin d'un chronomètre ou la réalisation d'une action.
- **Environnement :** Garantir un placement aléatoire des dinosaures et des déplacements fluides qui s'adaptent automatiquement à la topographie de n'importe quel terrain.

# 2 Conception

Les principales règles du jeu ainsi que le choix des fonctionnalités pour le déroulement de la partie ou encore l'interface graphique de *JurassicWar* sont détaillées dans cette section.

## 2.1 Principales règles du jeu

*JurassicWar* se compose de deux équipes, contenant chacune trois dinosaures. Au départ d'une partie, tous les dinosaures de la première équipe se trouvent à gauche du terrain et tous les dinosaures de la seconde équipe à droite de ce même terrain comme le montre la figure 1. L'objectif du jeu est de tuer les dinosaures de l'équipe adverse. La partie s'achève lorsque tous les dinosaures d'une équipe sont morts. L'équipe comportant des membres toujours vivants remporte le jeu. Pour tenter de gagner la partie, les dinosaures peuvent se déplacer et recourir à diverses armes telles que l'arc, la bombe ou encore le revolver. De plus, pour circuler plus rapidement sur la zone de jeu ou accéder à certains endroits difficiles d'accès, les dinosaures disposent d'un grappin. Par ailleurs, les dinosaures jouent chacun leur tour. Ils disposent d'un temps limité par tour pour agir et ne peuvent utiliser qu'un seul outil par tour. De plus, ils doivent se plier à un certain nombre de règles pour sur-

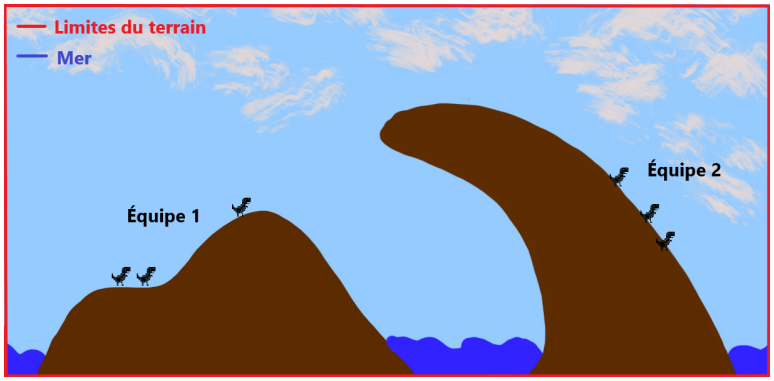



FIGURE 1 – Illustration des équipes sur le terrain et des zones inaccessibles

vivre le plus longtemps possible. Premièrement, un dinosaure possède cent points de vie au départ d'une partie. Il meurt s'il a perdu tous ses points de vie, s'il tombe dans la mer ou heurte une extrémité du terrain. Ces zones, que les dinosaures doivent éviter, sont représentées sur la figure 1. Deuxièmement, si le dinosaure touche accidentellement un autre dinosaure de son équipe en tirant avec une arme, le dinosaure visé perd le même nombre de points de vie que pour un coup frappant un dinosaure de l'équipe adverse. Le jeu nécessite ainsi de l'adresse pour parvenir à tuer l'équipe opposée mais aussi pour ne pas blesser accidentellement un membre de son équipe.

## 2.2 Foncionnalités prévues

Des choix ont été effectués concernant les fonctionnalités à inclure dans *JurassicWar*. Il s'agit d'un jeu à deux joueurs qui n'offre pas l'opportunité de mener une partie individuellement. Il utilise une matrice pour réprésenter le terrain sur lequel le jeu se déroule. Les joueurs disposent de quatre armes de tir, respectivement l'arc, l'arbalète, le revolver et le fusil ainsi que d'une arme explosive, la bombe. Le système de tir propose une prévisualisation de la trajectoire qu'il est possible de modifier avant de valider l'envoi du projectile. Le jeu n'inclut pas uniquement des armes mais aussi d'autres outils comme une potion de soin et un grappin. Pour recourir à ces différents outils, le joueur appuie sur les touches du clavier. En outre, *JurassicWar* intègre une interface utilisateur. En effet, il propose un affichage du terrain, des dinosaures et des armes en deux dimensions comme le souligne la figure 1. Il propose également un menu principal et un menu de changement d'armes dont l'aspect est visible sur la figure 2. De surcroît, le jeu indique en temps réel le nombre de points de vie restant des dinosaures sur la fenêtre du jeu.

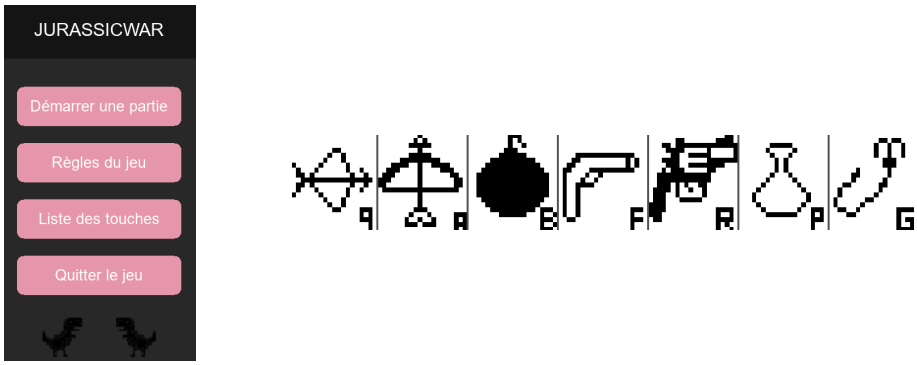



FIGURE 2 – Menus proposés dans *JurassicWar*

# 3 Organisation du travail

Cette partie montre la méthodologie structurelle adoptée pour la réalisation du projet JurassicWar, allant de la coordination du groupe à la gestion technique.

## 3.1 Répartition des tâches

Tout d'abord la conception du projet a été séparée en trois axes majeurs. La gestion des tours et la physique complexe des explosifs ont été confiées à Hannah. Le traitement du terrain matriciel, les déplacements et le système de grappin ont été pris en charge par Solène. Enfin, le développement des armes de tir, du système de soin, de l'inventaire et de l'interface utilisateur a été assuré par Romane. Cette répartition permet de pouvoir travailler séparément sans empiéter sur le programme de l'autre afin de ne pas perdre de temps et de ne pas se couper mutuellement le travail. Cette méthode apporte de la liberté au code et une plus grande liberté d'imagination qui aurait pu bloquer si deux personnes travaillait sur le même sujet mais sans avoir les mêmes idées. Ensuite la documentation étant nécessaire, chaque membre du groupe a eu la charge de documenter ses propres fonctions. Cette manière de faire garantit que chaque ligne de code est explicitée facilitant la relecture. L'organisation a été rythmée par plusieurs types de rencontres. Des réunions de répartition initiales ont permis de définir les structures communes. Par la suite, des réunions hebdomadaires de "micro-planning" ont été instaurées. Celles-ci servaient à ajuster les objectifs en fonction des blocages techniques rencontrés (par exemple, l'adaptation des trajectoires de tir au relief du terrain). Ces échanges ont assuré une synchronisation constante entre les différents modules.

## 3.2 Planning prévisionnel

Le projet a été planifié sur une période de douze semaines. Des semaines 1 à 3 l'accent a été mis sur la définition des structures de données (t_dino, t_tir, t_map) et sur la conversion du terrain image en matrice. Sans ces bases, aucun développement parallèle n'était possible. Ensuite de la semaine 4 à la semaine 7 l'implémentation des fonctions était vitale, surtout celles des détection des collisions, du lancement des armes, des déplacements et du changement de tour. Le planning montre bien une forte imbrication des tâches nécessitant des tests réguliers. Enfin les dernières semaines ont été consacrées à l'interface, la mise en commun des fonctions crées séparément et au réglage des dernières erreurs.
L'image ci-dessous (voir figure 3) illustre le diagramme de Gantt du projet. Il est possible d'y observer les marges de sécurité prévues pour les tâches critiques, comme la gestion des rebonds de bombes, les déplacements ou les

tirs d'armes à feu, qui ont nécessité plus de temps de test que prévu.

| | | | | | |
|---|---|---|---|---|---|
| **Etape 1** | **Initialisation** | | | | |
| | Répartition des tâches | | Hannah, Romane, Solène | 22/01/26 | 30/01/26 |
| | Création diagramme de Gantt | | Hannah, Romane, Solène | 27/01/26 | 30/01/26 |
| | README | | Romane, Hannah | 11/03/26 | 05/04/26 |
| **Etape 2** | **Creation des managers** | | | | |
| | Réunions de répartition des tâches | | Hannah, Romane, Solène | | |
| | Réunion 2 (TP Projet 1) | | Hannah, Romane, Solène | | |
| | Réunion hebdomadaire (micro planning) | | Hannah, Romane, Solène | | |
| | | | | | |
| **Etape 3** | **Codage & Test** | | | | |
| Tâche 1 | Définir les structures | | Hannah, Romane, Solène | 27/01/26 | 08/04/26 |
| Tâche 2 | Coder les fonctions d'initialisation | | Hannah | 27/01/26 | 14/02/26 |
| Tâche 3 | Création de l'image du terrain | | Solène | 31/01/26 | 01/02/26 |
| Tâche 4 | Fonction de conversion d'images en matrices | | Solène | 07/02/26 | 15/02/26 |
| Tâche 5 | Fonction mise à jour terrain | 1,2,4 | Solène | 05/02/26 | 25/03/26 |
| Tâche 6 | Fonctions rebonds | 1,2 | Hannah | 05/02/26 | 09/04/26 |
| Tâche 7 | Fonctions bombes | 1,2,5,6 | Hannah | 05/02/26 | 09/04/26 |
| Tâche 8 | Fonctions de collisions/frontières (décor) | 1,4 | Romane | 07/02/26 | 08/04/26 |
| Tâche 9 | Fonctions d'affichage du jeu | 1,2,4,5 | Hannah, Romane, Solène | 14/02/26 | 17/02/26 |
| Tâche 10 | Fonction placement dinosaures | 1,2,4,8,5 | Romane | 15/02/26 | 09/04/26 |
| Tâche 11 | Fonctions de déplacement | 1,2,4,8 | Solène | 25/02/26 | 09/04/26 |
| Tâche 12 | Fonctions d'armes de tir | 1,2,5,8 | Romane, Solène | 17/03/26 | 09/04/26 |
| Tâche 13 | Fonction menu principal | 1,2 | Hannah | 26/03/26 | 06/04/26 |
| Tâche 14 | Fonction changement de joueurs/dinosaures/tours | 1,2 | Hannah | 20/03/26 | 24/03/26 |
| Tâche 15 | Création des images des armes | 1,2 | Romane | 28/03/26 | 31/03/26 |
| Tâche 16 | Fonctions inventaire (outils, armes) | 1,2 | Romane | 31/03/26 | 08/04/26 |
| Tâche 17 | Fonctions d'affichage des points de vie | 1,2 | Romane | 31/03/26 | 08/04/26 |
| Tâche 18 | Fonction de soin | 1,2 | Romane | 01/04/26 | 09/04/26 |
| Tâche 19 | Fonctions pour le grappin | 1,2 | Solène | 03/04/26 | 13/04/26 |
| Tâche 20 | Fonctions de libération du main | | Romane/Hannah | 03/04/26 | 13/04/26 |
| | | | | | |
| Tâche autre | Documentation Doxygen | | Hannah,Romane,Solène | 07/02/26 | 13/04/26 |
| | Makefiles | | Hannah, Romane, Solène | | |

FIGURE 3 – Planning prévisionnel de JurassicWar

### 3.3 Outils de programmation utilisés

Le recours à des outils de développement professionnels a permis de maintenir une stabilité logicielle tout au long du cycle de production.
L'outil "git" a permis la fusion des codes provenant de trois développeurs différents et a permis à chaque personne de pouvoir coder tout en pouvant s'inspirer du code des autres. Ensuite la documentation avec "doxygen" est générée automatiquement à partir des commentaires formatés dans le code source. Exemple : l'utilisation des balises @param et @return dans le fichier fonctionsTirs.c permet d'obtenir une notice technique claire sans rédaction manuelle supplémentaire. Cela est un gain de temps et permet un meilleur visuel à la fois pour les personnes qui essayent de comprendre le code et à la fois pour les créateurs qui cherchent à retrouver l'utilité d'une fonction ou d'une strcuture, que ce soit durant le projet ou dans le futur. Après "GDB" a été utilisé pour stopper l'exécution du programme lors d'une erreur de segmentation afin d'identifier la ligne fautive et "Valgrind" a servi à détecter les fuites de mémoire. Exemple : la vérification que chaque texture SDL chargée en mémoire est correctement libérée à la fermeture du jeu. Enfin Un fichier

"Makefile" a été rédigé pour compiler l'ensemble du projet en une seule commande ce qui est un gain de temps énorme pour tester toutes les fonctions de tests crées dans différents fichiers. Exemple : la commande "make" gère la compilation séparée des fichiers gestion_zones.c et placer_dinos.c avant de les lier dans l'exécutable final.

# 4 Developpement

Les éléments techniques nécessaires à la mise en place du jeu sont détaillés dans les volets suivants. La modélisation du terrain, la gestion des déplacements ainsi que l'ensemble des actions réalisables au cours d'une partie y sont notamment décrites.

## 4.1 Représentation du terrain par une matrice

Afin de gérer les collisions et les interactions, le terrain est modélisé par une matrice de structures *t_case*. Chaque cellule de cette matrice, contient une constante identifiant la nature de l'élément présent : `EAU`, `AIR`, `TERRE` ou l'un des six dinosaures (`D1` à `D6`). Pour garantir l'adaptabilité du terrain à n'importe quel environnement, la fonction *conversionMatrice* convertit une image simplifié du terrain, en matrice pour créer un masque de collision :

- **Bleu :** représente l'eau ;
- **Marron :** représente la terre ;
- **Blanc :** représente l'air.

L'algorithme parcourt l'image pixel, par pixel (lignes et colonnes) et initialise la matrice. Une marge de tolérance sur les composantes RVB a été intégrée pour éviter des légères variations de couleur et assurer une détection robuste des éléments du décor.

## 4.2 Positionnement et mobilité des dinosaures

Au travers de cette partie nous allons voir comment l'utilisation des tableaux de coordonnées nommés nuages de points ont permis des actions et méthodes permettant aux dinosaures de se placer et se déplacer.

Tout d'abord pour modéliser le relief du terrain praticable, les coordonnées $(x, y)$ de la ligne de crête sont enregistrées dans un nuage de points. Un nuage de points est associé pour chaque colline. Le jeu est limité à 5 nuages de points par carte. La trace du nuage de points est enregistée au même dimension que l'image du jeu, voir figure 4.
Afin de connaître la position du dinosaure par rapport aux différentes collines, l'identifiant du nuage de points et l'indice de position dans ce nuage sont mémorisés dans la structure *t_dino*. L'identifiant 0 indique qu'il s'agit

de la zone de déplacement la plus à gauche, tandis que $n-1$ ($n$ le nombre de nuages) signifie qu'il sagit de la dernière colline, celle la plus à droite. Ce système d'indexation permet au dinosaure de suivre les courbes du terrain de manière ordonnée.

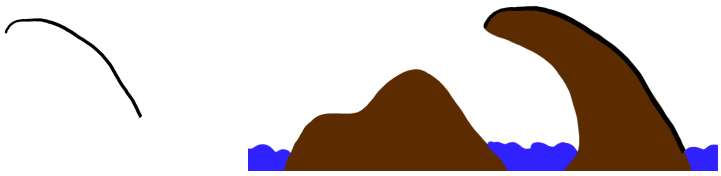
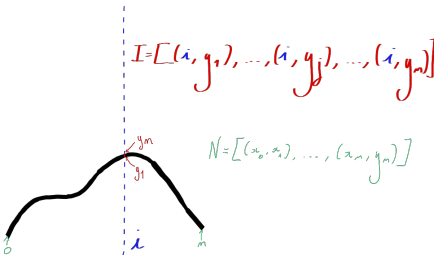


FIGURE 4 – schéma de la définition du nuage de points

Pour générer le nuage de points, la fonction *nuage_de_points* parcourt sur la largeur une image représentant le tracé d'une colline. Elle fait la moyenne $m$ des pixels noirs rencontrés, stockés dans un tableau $I$, à l'indice $i$. Le calcul de la moyenne $m$ est ensuite enregistré dans un tableau $N$ de *t_coordonnee* avec $(x = i, y = m)$. Comme le tracé est forcément continue, alors les points $x$ sont reliés les uns aux autres de façons croissante et continue, voir image : figure 5 accompagné du calcul de $(x_i, y_i)$.




$$I = [(i, y_1), ..., (i, y_j), ..., (i, y_n)]$$

$$N = [(x_1, y_1), ..., (x_i, y_i), ..., (x_n, y_n)]$$

$$(x_i, y_i) = (i, m = \sum_{k=0}^{n-1} \frac{y_k}{n})$$

FIGURE 5 – schema du calcul du tableau de nuage de points

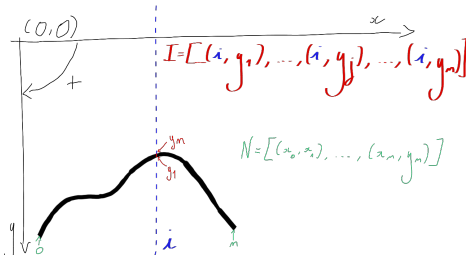
Le placement initial des unités est une étape importante pour assurer la non redondance de JurassicWar. Plutôt que d'imposer des positions fixes, le jeu a un système de génération de zones de départ basé sur l'analyse topographique du terrain.

Le terrain est divisé en deux zones pour le placement des dinosaures : le premier tiers, à gauche pour l'équipe 1 et le dernier tiers, à droite pour l'équipe 2. Chaque zone dispose de 3 dinosaures qui correspond à une équipe. La fonction *generer_catalogue_depuis_nuage* parcourt le nuage de points (représentant la surface du sol) pour extraire des coordonnées viables. Pour éviter que les dinosaures ne se chevauchent dès le départ, la fonction *distance*

calcule l'écart euclidien entre chaque nouveau point et ceux déjà sélectionnés. Si la distance est inférieure à 30 pixels, le point est rejeté. Ensuite afin de prévenir tout risque de boucle infinie (dans le cas d'un terrain trop accidenté ou exigu), une variable "securite" limite l'algorithme à 5000 tentatives. Enfin le programme remplit un catalogue de 10 zones potentielles par équipe, permettant de multiples configurations de départ possibles.
Une fois le catalogue généré, la fonction *placer_une_equipe* place trois dinosaures dans chaque équipe. Chaque dinosaure reçoit un identifiant unique qui sera injecté dans la matrice de jeu. Cette injection dans la matrice se fait via la fonction *remplir_matrice_dino*. Cette étape est très importante car la matrice est ensuite utilisée en permanence afin de jouer. Le dinosaure, qui est représenté par un carré de 30x30 pixels, remplace les valeurs de la matrice par l'ID du dinosaure mais avant d'écraser une case de la matrice, le dinosaure stocke la valeur précédente (terre ou air) dans son tableau mémoire. Cette précaution est essentielle pour "restaurer" le décor lorsque le dinosaure se déplace ou meurt, évitant ainsi de laisser des trous vides dans le terrain.
Enfin, l'affichage avec les fonctions *afficherDinos* et *afficherDinosAvecJeu*. Une distinction a été faite avec la deuxième fonction pour prendre en compte le décalage vertical de 100 pixels lié à l'interface utilisateur. Cela permet de séparer la logique de collision (fondée sur les coordonnées réelles de la matrice) de la logique visuelle (adaptée à la fenêtre SDL)
Afin de créer un mouvement réaliste lorsque le dino marche, la fonction *regression* prend un échantillon des dix points les plus proches du dinosaure et calcul le coefficient directeur $\alpha$ qui indique le degré de la pente à la position $(x_i, y_i)$, voir image : figure 6régression accompagné du calcul de $\alpha$. La fonction *marcher* adapte la vitesse du pas en fonction de $\alpha$. En effet si $\alpha > 0$ le dinosaure descend vers la gauche et monte vers la droite et inversement si $\alpha < 0$. Sinon, le dinosaure marche à la vitesse de base. On trouve ainsi le pas qui permet d'incrémenter ou de décrémenter plus ou moins vite l'indice du dino dans le nuage de points avec la formule suivante :

$$pas = VITESSE_BASE \cdot (1 + sens \cdot (a * 0,5)) \tag{1}$$

Si le dinosaure sort des extrémités du tableau, alors il se noie ou il tombe sur une nouvelle colline. La fonction *replacementNuage*, calcule l'écart entre la position x du dinosaure et la coordonnée $x'$ du point de bordure d'un nuage, le plus proche du dinosaure. Le nouvel indice dans le nuage devient $0 + ecart$, si le dinosaure est à côté du début du nouveau nuage, sinon il est égale à $n - 1 - ecart$, $n - 1$ étant le dernier indice du nuage.
Par sécurité, elle intègre également une règle interdisant le franchissement de dénivelés verticaux trop importants. Si aucun nuage n'est trouvé, le dinosaure tombe dans l'eau et se noie.




$$a = \frac{\sum_{k=i-5}^{i+4}(x_k - x_i)(y_k - y_i)}{(x_k - x_i)^2}$$

FIGURE 6 – schema du calcul de la pente $\alpha$

Pour que le dinosaure saute, la flèche du haut doit être cliquée. La vitesse de saut, stockée dans *t_ dino*, du dinosaure est initinialisée à **FORCE_SAUT**, qui est une valeur négative qui propulse le dinosaure vers le haut. La vitesse de saut est alors ajoutée à la position $y$ du dinosaure. Puis, la vitesse du saut est ralentie grâce à l'accumulation de la constante **GRAVITE** tour par tour, jusqu'à ce que le dinosaure retombe. Le saut prend fin, lorsque la position $y$ du dinosaure est supérieure ou égale à la position $y'$ du nuage de points actif à *indice_ nuage* du dinosaure.
Pour déplacer, à gauche, ou à droite, **lors d'un saut**, l'indice du nuage du dinosaure est incrémenté ou décrémenté. Si en sautant le dinosaure sort du nuage, alors il y a deux possibilités, soit un nouveau nuage de point est trouvé avec la fonction *replacementNuage* et le saut reprend normalement, soit il tombe dans l'eau.

## 4.3 Système d'actions et outils à disposition du joueur

Cette partie rend compte de la manière dont s'enchaînent les actions dans *JurassicWar* à travers la mécanique des changements de tours mais aussi de la nature des actions possibles dans le jeu. Un dinosaure peut en effet user de différents outils tels que la bombe, les armes de tirs ou encore le grappin.

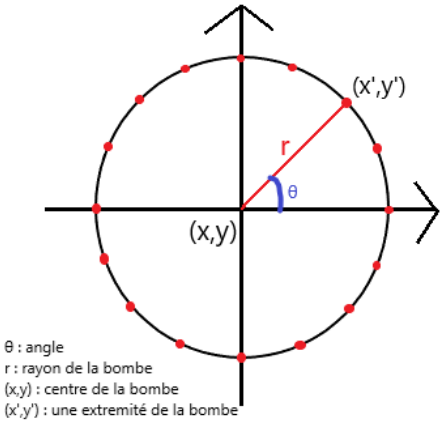
### 4.3.1 Mécanique des changements de tours

Premièrement, les dinosaures jouent chacun leur tour, c'est-à-dire que l'utilisateur n'a pas la possibilité de choisir avec lequel de ses trois dinosaures il souhaite agir à un tour donné. L'instauration de cette règle augmente d'une part la complexité du jeu, mais d'autre part le système des changements de tours, puisque l'alternance entre tous les dinosaures doit être correcte même lorsqu'un ou plusieurs d'entre eux meurent. Afin de mettre en œuvre les changements de tours, une fonction *tourSuivant* a été crée. Elle s'appuie sur l'identifiant associé à chaque dinosaure pour déterminer le dinosaure qui peut jouer. Cependant, comme les dinosaures de la première équipe sont numérotés de un à trois et ceux de la seconde équipe de quatre à six, à la suite du premier dinosaure, c'est au quatrième dinosaure de jouer et non pas au

second dinosaure pour garantir l'alternance entre les deux équipes. Ceci nécessite de mémoriser deux identifiants : celui du dinosaure qui vient d'agir et celui du dinosaure ayant agi précédemment. Ces identifiants sont stockés dans uns structure *t_ tour* qui contient également le numéro de l'équipe qui vient de jouer et un compteur du nombre de tours effectués. Par ailleurs, la fonction *tourSuivant* doit tenir compte de l'état d'un dinosaure dans la mesure où un dinosaure mort ne peut plus jouer. Ainsi, une fois que la fonction a choisi un dinosaure, elle vérifie qu'il est vivant. Dans le cas contraire, elle sélectionne un autre dinosaure jusqu'à en trouver un encore dans la partie. Une sécurité a été ajoutée pour que la fonction ne renvoie aucun dinosaure en fin de partie, c'est à dire lorsque tous les dinosaures d'un joueur ont péri.

### 4.3.2 Infliger un coup mortel : bombe

Deuxièmement, au cours d'une partie, les joueurs peuvent faire appel à une arme mortelle : la bombe. Contrairement aux armes de tirs évoquées dans la section suivante, la bombe tue instantanément le dinosaure qu'elle heurte. Néanmoins, cette arme peut uniquement être employée si le dinosaure dispose d'une place suffisante à côté de lui pour la positionner. L'objectif consiste à apporter de la difficulté supplémentaire mais aussi un intérêt à utiliser chaque arme disponible dans le jeu. De surcroît, cette arme se caractérise par sa capacité à rebondir sur le terrain. Cette capacité de la bombe à rebondir induit l'utilisation de plusieurs notions mathématiques dans le code. Pour faciliter l'application de ces notions, la bombe est représentée par une structure *t_ bombe* comportant notamment le rayon et le vecteur vitesse du projectile, c'est à dire la vitesse sur l'axe horizontal et vertical de la bombe. Tout d'abord, la mise en place des rebonds nécessite un système de collision avec le sol précis en raison du relief particulier du terrain. Le terrain étant formé de collines et de cratères aux courbures arrondies, une détection de collisions limitée aux quatre directions haut, gauche, bas et droite ne suffit pas et manque les impacts en diagonale. En conséquence, pour élargir le nombre de directions étudiées lors des collisions, la fonction *collisionTerrainBombe* fait recourt aux formules des coordonnées polaires pour analyser une multitude de points sur le contour de la bombe. Les points analysés sont représentés en rouge sur la figure 7 accompagnés de la formule pour calculer leurs coordonnées. En cas de collision avérée entre un ou plusieurs de ces points et la terre, une seconde étape consiste à étudier la pente du terrain de manière à ce que la bombe rebondisse perpendiculairement au sol. Pour cela, les valeurs dans la matrice des points voisins au lieu d'impact sont examinées et les pixels contenant de la terre sont intégrés dans le calcul d'un gradient à la surface du terrain, c'est à dire d'un vecteur qui pointe vers la direction où la densité de la terre est la plus forte. Un exemple de gradient pouvant être établi par la fonction *collisionTerrainBombe* est représenté sur la figure 8. Dans le calcul du gradient, il est donné un poids important aux points jouxtant l'impact et




$$x' = x + r\cos(\theta)$$

$$y' = y + r\sin(\theta)$$

FIGURE 7 – Schéma des coordonnées polaires appliqué à la bombe

un poids plus faible aux voisins plus éloignés afin de rendre compte au mieux du relief et pour qu'aucune éventuelle irrégularité du terrain ne fausse les résultats. De plus, dans le cas où toute une partie de la bombe est en contact avec le sol, le gradient est déterminé pour chaque point de collision. En prenant l'opposé de chacun de ces vecteurs, une famille de vecteurs normaux est formée, c'est à dire un ensemble de vecteurs perpendiculaires au terrain. Le vecteur normal finalement choisi pour le rebond constitue une moyenne de tous ces vecteurs. La bombe suit la direction de ce vecteur pour s'éloigner

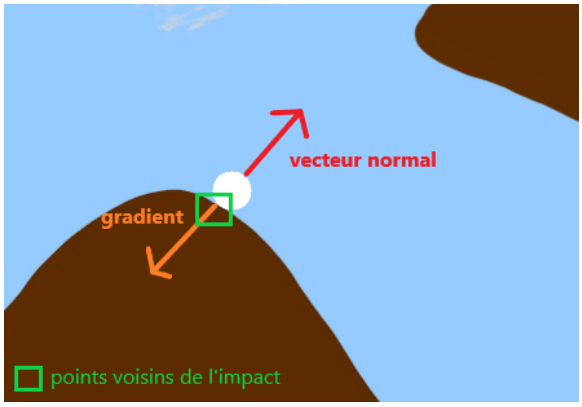



FIGURE 8 – Exemple d'un gradient et d'un vecteur normal associé à la bombe en un point donné du terrain

du terrain comme le recapitule la figure 8. Par ailleurs, le choc n'impacte pas uniquement la trajectoire de la bombe mais également sa vitesse. La nouvelle vitesse de la bombe suite au rebond est obtenue par la formule de réflexion par rapport à un plan :

$$\vec{v}' = \vec{v} - 2\,(\vec{v}\cdot\vec{n})\,\vec{n} \tag{2}$$

où $\vec{v}$ désigne le vecteur vitesse de la bombe, $\vec{v}'$ le vecteur vitesse après rebond et $\vec{n}$ le vecteur normal. Pour procurer plus de réalisme au rebond et décrire le ralentissement se produisant dans le monde réel lorsqu'un projectile percute un objet, un coefficient de restitution intervient dans la formule 2 pour moduler la valeur du facteur deux. Ce coefficient varie entre zéro et un, où un signifie un rebond parfait et zéro une absence totale de rebond. Enfin, concernant la vitesse et la trajectoire de la bombe lorsqu'elle ne rencontre pas d'obstacle sur son chemin, celles-ci reposent sur une approximation des équations de mouvement suivantes :

$$\begin{aligned} x(t) &= x_0 + v_{x0}\, t \\ y(t) &= y_0 + v_{y0}\, t + \frac{1}{2} g t^2 \end{aligned} \tag{3}$$

où $(x_0, y_0)$ désignent les coordonnées à l'instant présent de la bombe, $(x, y)$ les nouvelles coordonnées calculées, $\vec{v} = (v_{x0}, v_{y0})$ le vecteur vitesse, $t$ le temps écoulé et $g$ la gravité. De la même manière que précédemment, l'introduction du temps et de la gravité dans l'équation a pour visée de reproduire plus fidèlement possible le comportement d'une bombe dans le monde réel.

### 4.3.3 Blesser un dinosaure : armes de tir

Après l'implémentation de la bombe, nous avons conçu un système d'armes à distances variées. L'objectif principal est de proposer une alternative à la bombe en introduisant une gestion balistique différenciée selon l'équipement choisi. Afin d'apporter du choix, quatre armes de tir ont été implémentées : l'arc, l'arbalète, le fusil et le revolver. Contrairement à la bombe, ces armes ne sont pas mortelles au premier contact mais retirent une quantité définie de points de vie. Pour améliorer le réalisme du jeu, chaque projectile possède des propriétés physiques propres définies dans la structure *t_arme* : Son nom, sa puissance de propulsion qui va déterminer la vitesse initiale du projectile. Un fusil aura une vélocité supérieure à celle d'une flèche. Sa vitesse maximale car chaque arme ne peut dépasser une certaine vitesse. Sa masse, un projectile lourd comme une flèche aura une courbe de chute plus marquée qu'une munition de revolver et sa dangerosité, c'est à dire le nombre de points de vie enlevés lors de l'impact.

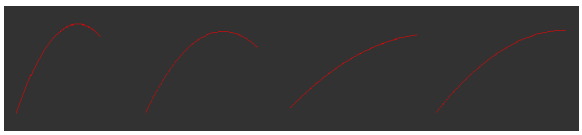


FIGURE 9 – Exemple de la trajectoire des quatres armes

L'utilisation d'une arme de tir se décompose en trois phases gérées par la fonction *viserArcher* : Tout d'abord le joueur choisit son arme via des touches

dédiées pour chacune d'entre elles, Q pour l'arc, A pour l'arbalète, F pour le fusil et R pour le revolver. Une fois l'arme en main, le dinosaure ne peut plus bouger afin de se concentrer sur son tir. Ensuite à l'aide des flèches directionnelles, l'utilisateur modifie le vecteur vitesse (tir→velo.u et tir→ velo.v). Afin de maintenir un jeu réaliste ces valeurs ont une vitesse maximale qui ne peut être dépassée. Enfin la fonction tracerTrajectoireTir permet d'afficher en temps réel une courbe prédictive en pointillés rouges. Cette simulation calcule les 30 prochaines positions théoriques du projectile en appliquant la gravité à chaque itération, offrant ainsi une aide à la visée. Une fois la touche Espace pressée, le projectile passe à l'état actif avec la fonction *mettreAJourVol*. À chaque frame, elle applique l'effet de la gravité selon la formule :

$$vy(t+1) = vy(t) + gravite * poids_projectile \tag{4}$$

Le vol du projectile est soumis à de nombreuses vérifications via différentes fonctions de détection comme *collisionFrontière* qui désactive le tir s'il sort des limites du terrain ; *collisionTerrain* et *collisionEau* qui analysent la matrice du terrain et si le projectile touche la terre ou l'eau le projectile s'arrête. Enfin la fonction *collisionDino*, le programme parcourt la matrice afin de vérifier si les coordonnées du tir correspondent au numéro d'un dinosaure. Une sécurité est rajoutée ici afin d'empêcher le tireur de se tirer dessus et de se blesser. Lorsqu'une collision avec un dinosaure adverse a lieu, la fonction

```
Collision avec le dino 4
Collision avec le dino 2
Collision avec le dino 6
Collision avec une la terre !
Collision avec une l'eau !
Collision avec une l'eau !
Collision avec une la terre !
Collision avec le dino 4
Collision avec le dino 6
Collision avec une la terre !
Collision avec une la terre !
Collision avec une la terre !
Collision avec une la terre !
Collision avec une l'eau !
Collision avec le dino 3
Collision avec le dino 6
Le Dino 6 est KO !
Collision avec une limite du jeu
Collision avec le dino 7
Collision avec une limite du jeu
Collision avec le dino 5
Collision avec le dino 2
Collision avec le dino 7
```

FIGURE 10 – Résultats suite à une collision

*appliquerDegats* intervient. Cette fonction récupère le pointeur sur la structure du dinosaure touché pour mettre à jour ses points de vie. Si les PV

tombent à zéro, le dinosaure meurt, sa texture est supprimée de l'affichage et ses coordonnées effacées de la matrice de jeu.
En opposition aux armes qui blessent, une potion de soin a été implémentée. Le joueur peut choisir, lors de son tour, d'utiliser une potion au lieu de tirer. Le soin est aléatoire : il peut redonner jusqu'à 20 points de vie, soit 20% de la vie totale (voir figure 11 et figure 12). Cette incertitude ajoute un élément de risque, le joueur devant choisir entre tenter d'achever un adversaire ou espérer un soin salvateur pour prolonger la survie de son équipe.

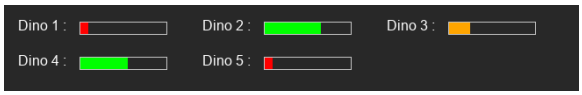



FIGURE 11 – Dinosaures blessés

FIGURE 12 – Résultats suite à un soin

#### 4.3.4 Autre méthode de déplacement : grappin

Pour utiliser le grappin, il faut appuyer sur la touche **G**. Le joueur choisit alors l'angle pour lancer le grappin à l'aide de la fonction *choixAngleLancer*. Elle ajoute ou enlève $0,5$ à l'angle en fonction des flèches appuyées. Puis grace aux fonctions trigonométriques, voir figure 7, elle projète les coordonnées $(x, y)$ de l'extrémité du segment de visée.
Le joueur appuie ensuite sur **ESPACE** pour lancer le grappin. Les fonctions *lancer* et *rappel* font appel à une **file** qui mémorise chronologiquement les points de la trajectoire lors du lancer, puis les restitue lors du rappel afin de tirer le dinosaure.
Si le joueur décide d'appuyer sur **ENTREE**, la traction se stoppe et le dinosaure se balance. La fonction *balancier* simule un mouvement de pendule autour de la queue de la **file**, utilisée comme pivot fixe. Elle calcule l'accélération et la vitesse d'oscillation pour transformer les coordonnées polaires 7 en positions cartésiennes. Le joueur peut alors accentuer le balancement avec les flèches directionnelles avant de stabiliser sa position finale.
Cependant si le dinosaure touche la colline avant d'appuyer sur **ENTREE**, il y a alors deux possibilités. Premier cas de figure, le dinosaure se situe en desous de la ligne de crête, la fonction *chute* est appelée, il peut alors atterir sur le nuage ou se noyer. Second cas de figure c'est *replacementNuage* qui est appelée pour replacer correctement le dinosaure sur la nouvelle colline.

## 4.4 Interface graphique et utilisateur

La visualisation à l'écran tous les éléments créés dans le cadre du projet tels que la matrice, le positionnement des dinosaures ou encore les outils, nécessite une interface graphique. Cette interface, développée grâce au module Simple DirectMedia Layer (SDL), constitue la base de l'interface utilisateur du jeu en intégrant divers menus et boutons qui facilitent la prise en main de *JurassicWar* par les joueurs.

### 4.4.1 Utilisation du module SDL

En premier lieu, la sélection du module SDL pour le projet se justifie par sa portabilité. Contrairement à d'autres librairies graphiques, SDL fonctionne à la fois sur Linux, Windows et macOS et est adapté au langage C. Il est principalement employé pour afficher le terrain, les dinosaures, les armes ainsi que divers menus explicités par la suite. Toutefois, il gère également les différentes actions du joueur tel qu'appuyer une touche du clavier pour choisir une arme et la trajectoire du tir dans les fonctions *choixHauteurLancer* et *viserArcher*, ou encore cliquer sur les boutons du menu principal dans la fonction *detecterEvenementsMenuPrincipal*. Sans oublier que les touches du clavier se veulent indispensables aux déplacements des dinosaures. Cependant, ce module représente également une contrainte puisqu'il requiert de gérer correctement la mémoire entre les différentes allocations et libérations que ce soit pour les fenêtres mais aussi pour les textures des dinosaures et des objets. En outre, obtenir un rendu propre lors du lancer de la bombe ou de l'envoi de flèches et des balles pendant les tirs exige des ajustements, notamment en ce qui concerne le délai d'affichage des images. D'un côté, un délai trop long engendre un visuel flou et peu esthétique pour l'utilisateur et d'un autre côté, un délai trop court consomme beaucoup de ressources. Par conséquent, cet intervalle de temps est directement lié à la vitesse des projectiles et donc aux calculs mathématiques liés à ceux-ci. Enfin, l'interface graphique de *JurassicWar* cumule une multitude d'images et de textes lorsque tous les élements du jeu, c'est-à-dire l'inventaire et le terrain sont réunis. Ceci impose un ordre d'affichage strict pour assurer la fiabilité de l'affichage. C'est un enjeu rencontré par les différentes versions des fonctions *afficherJeu*.

### 4.4.2 Création d'un menu principal

En second lieu, un menu principal a été conçu dans *JurassicWar* afin de fournir une interface intéractive centralisant l'accès aux fonctionnalités essentielles du jeu. Il comporte quatre onglets permettant de démarrer la partie, de consulter les principales règles du jeu, d'accéder à la liste des touches du clavier et de quitter le programme. Les règles instaurées dans la partie ainsi que la liste des touches du clavier correspondant à chaque

action figurent comme indispensables dans le menu dans la mesure où ces informations sont inconnues au joueur lors de la découverte du jeu. Ces informations sont placées dans des fenêtres distinctes accessibles depuis des boutons. Leur affichage repose sur le module SDL, abordé dans la section précédente et employé dans plusieurs fonctions telles que *tracerBouton* ou encore *afficherMenuPrincipal*. De même, la détection des clics dans le menu est réalisé par SDL. À propos du texte contenu dans les fenêtres, celui-ci est stocké dans des fichiers, lus lorsque l'utilisateur demande l'ouverture d'une page par le biais de la fonction *lireInfosFichier*. Le texte, rédigé en français et contenant de nombreux caractères accentués, a nécessité de tenir compte de l'encodage en UTF-8 des caractères, notamment pour obtenir un rendu correct avec SDL. L'intégration du menu principal avec le reste du code du jeu a consisté à découper le code déjà existant en fonctions et à adapter la structure du programme principal. Dans un premier temps, le programme central invoque la fonction *ouvrirMenuPrinc* qui initialise le menu principal et sollicite toutes les fonctions liées au menu. Puis, dans un second temps, en cas d'une action de l'utilisateur dans ce sens, la fonction *lancerPartie* s'exécute. Cette fonction orchestre tout le fonctionnement global du jeu et fait appel à toutes les fonctions rédigées dans le cadre du projet.

### 4.4.3 Faciliter l'utilisation du jeu pour l'utilisateur

Pour finir l'adaptation en cours de partie repose sur la capacité de l'interface à transmettre des informations complexes de manière instantanée. Cette fluidité de jeu tourne autour de deux grands axes : la gestion des ressources via un inventaire dynamique et le suivi en temps réel de l'état des unités.

Premièrement, la mise en œuvre d'un inventaire situé en bordure supérieure gauche de l'écran centralise les outils à disposition (voir figure 13). Cette interface, gérée par la fonction *afficherInventaire*, segmente l'espace en 7 cases distinctes associées aux textures des objets (Arc, Arbalète, Fusil, Revolver, Bombe, Grappin ou Potion). Afin de supprimer la nécessité de consulter une notice externe durant l'action, chaque image intègre une lettre de rappel. Ce marquage visuel indique directement la touche du clavier dédiée à l'activation de l'objet. La conception de cet inventaire repose sur un calcul dynamique de la largeur des cases en fonction de la constante *LARGEUR_INVENTAIRE* et du nombre d'objets, assurant une adaptabilité aux futures évolutions du catalogue d'armes. L'usage de *SDL_RenderDrawRect* délimite chaque emplacement par une bordure grise, renforçant la clarté pour l'utilisateur. Deuxièmement, le suivi de l'état de santé des dinosaures constitue un facteur stratégique important. Le menu des points de vie créé par les fonctions *afficherMenuPVDinos* et *afficherBarrePV*, offre une idée de l'avancée de la partie. L'interface regroupe les informations sur deux lignes, une par équipe (voir figure 13). Chaque dinosaure dispose d'une jauge de santé dont la cou-

leur évolue selon le nombre de points de vie restants. Le vert indique une santé supérieure à 50%, l'orange un état critique supérieur à 20% et le rouge un seuil d'urgence inférieur à 20%. Ce changement alerte l'utilisateur sans qu'une lecture précise des valeurs numériques ne soit utilisée. De plus, la disparition immédiate de la jauge lors du décès d'un dinosaure signale une perte importante du jeu. Enfin l'optimisation des performances d'affichage des textes et des jauges traité par la fonction *afficherMenuPVDinosOp* avec l'utilisation d'un système de cache de textures (*t_ texte_ cache*) limite les opérations coûteuses de rendu textuel à chaque cycle de la boucle de jeu. En stockant en avance les étiquettes de chaque dinosaure, le programme réduit la consommation des ressources processeur tout en maintenant une interface utilisateur réactive et informative.

FIGURE 13 – Système de jauges de santé dynamiques avec code couleur représentatif de l'état des unités et inventaire des armes.

# 5 Résultats et Conclusion

À l'issue du projet, un logiciel fonctionnel a été obtenu. Les joueurs peuvent lancer une partie, déplacer les dinosaures et utiliser tous les outils indiqués dans le cahier des charges initial. De plus, les interfaces graphiques prévues, c'est à dire le menu principal et l'inventaire ont été implémentées. Cependant, certaines fonctionnalités comme la possibilité pour l'utilisateur de choisir le terrain de jeu dans lequel il souhaite mener une partie ou encore la mise en place d'une limite de temps pour réaliser chaque action n'ont pas été mises en œuvre.

Quant au planning des tâches imposé pour le projet, il n'a pas été respecté. D'une part, les éléments requérant des notions mathématiques complexes comme la gestion des déplacements ou des rebonds de la bombe ont été retardés par la survenue de bugs nécessitant des corrections importantes. D'autre part, l'étape d'assemblage du code réalisé par chaque membre du groupe pour former la version finale du jeu avait été omise de la liste des tâches établies au début du projet. En outre, la charge importante de travail représentée par le cursus de double licence a également retardé le projet.

Une amélioration possible pour le projet consisterait en l'adaption de l'interface graphique aux différentes résolutions d'écran dans la mesure où la page de jeu est trop imposante pour certains écrans. Par ailleurs, une optimisation des calculs effectués dans le code rendrait le programme plus fluide.

La réalisation du projet a permis à notre groupe de développer de nouvelles compétences. Premièrement, la conception du jeu a exigé une organisation rigoureuse ainsi qu'une communication efficace entre les membres pour éviter des malentendus. Dexièmement, une documentation claire et structurée est apparue comme un élément central pour la compréhension et la réutilisation du code réalisé par les autres membres du projet. Enfin, la création de *JurassicWar* a constitué une occasion concrète de mettre en pratique les notions d'algorithmique étudiées préalablement.

# 6 Bibliographie

## Références

[1] Solène Orieux, Romane Saint-Léger, Hannah Sergent. *Projet L2 JurassicWar*. Disponible à : `https://github.com/romanesl72/Projet_L2_JurassicWar`, 2026.

## Glossaire

**gradient** Vecteur qui pointe dans la direction où une fonction mathématique augmente le plus vite. 12

**gravité** Force qui attire les objets vers le sol. 14

**réflexion** Description du changment de direction la vitesse d'un objet lorsqu'il heurte une paroi. 13

**régression** Méthode d'estimation consistant à déterminer une fonction minimisant l'écart entre les prédictions d'un modèle et les données observées. 10

**Simple DirectMedia Layer (SDL)** Bibliothèque multimédia permettant de gérer l'affichage, le son et les entrées dans les jeux vidéo. 17

**vecteur vitesse** Vecteur indiquant la direction et la rapidité du mouvement d'un objet. 12

# 7 Annexes

## 7.1 Jeux de tests

Afin de vérifier la comportement de certaines fonctions, des jeux de tests ont été effectués. Pour chaque fonction vérifiée, les cas de tests valides ou invalides sont énumérés dans des tableaux.

**Cas de test de la fonction mettreAjourVol dans le fichier test_vol.c** (table 1 et table 2).
`mettreAJourVol(t_tir *tir, int matrice[HAUTEUR_TERRAIN][LARGEUR_TERRAIN], float gravite, int id_tireur);`

| **Cas de test valides** | **Données** | **Résultats attendus** |
|---|---|---|
| Vol libre | Tir actif, pas d'obstacle | Retourne 0, position mise à jour, actif reste à 1 |
| Sortie Limites | Tir aux coordonnées (-1, -1) (en dehors du jeu) | Retourne -2, actif passe à 0 |
| Collision Terre | Matrice[y][x] = TERRE | Retourne 1 (TERRE), actif passe à 0 |
| Collision Eau | Matrice[y][x] = EAU | Retourne 2 (EAU), actif passe à 0 |
| Collision Dino | Matrice[y][x] = D2, id_tireur = 1 | Retourne D2, actif passe à 0 |
| Auto-Collision | Matrice[y][x] = D1, id_tireur = D1 | Retourne 0, actif reste à 1 |
| Tir Inactif | tir->actif = 0 | Retourne 0 |

Table 1 – Tableau des cas de test valides de la fonction mettreAJourVol

| **Cas de test Invalides** | **Données** | **Résultats attendus** |
|---|---|---|
| Pas assez d'arguments | (tir, matrice, gravite) | Erreur, données Invalides |
| Mauvais type d'arguments | (tir, gravite, matrice, tir) | Erreur, données Invalides |

Table 2 – Tableau des cas de test valides de la fonction mettreAJourVol

**Cas de test de la fonction recupererDinoNumero dans le fichier testFonctionsStructJoueur.c** (table 3 et table 4).

```
t_dino * recupererDinoNumero(t_joueur * equipe1, t_joueur * equipe2,
t_case numero);
```

| **Cas de test valides** | **Données** | **Résultats attendus** |
|---|---|---|
| Dinosaure trouvé dans l'équipe 1 | equipe1 = (D1, D2, D3), equipe2 = (D4, D5, D6), D2 | pointeur sur le dinosaure D2 |
| Dinosaure trouvé dans l'équipe 2 | equipe1 = (D1, D2, D3), equipe2 = (D4, D5, D6), D4 | pointeur sur le dinosaure D4 |
| Dinosaure présent dans les deux équipes | equipe1 = (D1, D2, D3), equipe2 = (D1, D5, D6), D4 | pointeur sur le dinosaure D1 de l'équipe 1 |
| Dinosaure deux fois présent dans la même équipe | equipe1 = (D1, D3, D3), equipe2 = (D1, D5, D6), D3 | pointeur sur le dinosaure D3 de l'équipe 1 possédant l'indice 1 |
| Dinosaure absent deux équipes | equipe1 = (D1, D2, D3), equipe2 = (D1, D5, D6), D4 | NULL |
| Numéro valant EAU, AIR ou TERRE | equipe1 = (D1, D2, D3), equipe2 = (D4, D5, D6), EAU | NULL |
| Numéro strictement supérieur à D6 valant 7 | equipe1 = (D1,D2,D3), equipe2 = (D4, D5, D6), 8 | NULL |
| Équipe 1 vide | equipe1 = (), equipe2 = (D4, D5, D6), D2 | NULL |
| Équipe 1 vide | equipe1 = (), equipe2 = (D4, D5, D6), D6 | D6 |
| Équipe 2 vide | equipe1 = (D1, D2, D3), equipe2 = (), D4 | NULL |
| Équipe 2 vide | equipe1 = (D1, D2, D3), equipe2 = (), D2 | D2 |
| Les deux équipes vides | equipe1 = (), equipe2 = (), D2 | NULL |

Table 3 – Tableau des cas de test valides de la fonction recupererDinoNumero

| **Cas de test invalides** | **Données** | **Résultats attendus** |
|---|---|---|
| Numéro non entier | equipe1 = (D1, D2, D3), equipe2 = (D4, D5, D6), 3.8 | données invalides |
| Numéro non eniter | equipe1 = (D1, D2, D3), equipe2 = (D4, D5, D6), ”essai” | données invalides |
| Équipe 1 n’existant pas | equipe1 = NULL, equipe2 = (D4, D5, D6), D2 | erreur de segmentation |
| Équipe 1 n’existant pas | equipe1 = NULL, equipe2 = (D4, D5, D6), D4 | erreur de segmentation |
| Équipe 2 n’existant pas | equipe1 = (D1, D2, D3), equipe2 = NULL, D4 | erreur de segmentation |
| Équipe 2 n’existant pas | equipe1 = (D1, D2, D3), equipe2 = NULL, D2 | erreur de segmentation |
| Équipe 1 et Équipe 2 n’existant pas | equipe1 = NULL, equipe2 = NULL, D2 | erreur de segmentation |

Table 4 – Tableau des cas de test invalides de la fonction recupererDinoNumero

**Cas de test de la fonction deplacement_dino dans le fichier deplacement.c** (table 5 et table 6).

```
void deplacement_dino(t_dino *dino, t_coordonnee **nuage, char *nomNuage[],
int nb_nuage, int *nb_pts, int matrice[HAUTEUR_TERRAIN][LARGEUR_TERRAIN],
t_joueur * equipe1, t_joueur * equipe2);
```

| **Cas de test Valides** | **Données (État + Action)** | **Résultats attendus** |
|---|---|---|
| Marche horizontale standard | `sautBooleen`=0, sur nuage + Touche `DROITE` | `pos.x` augmente, `indice_nuage` augmente, `sens`=`DROITE`. |
| Initialisation d'un saut | `sautBooleen`=0, `wait`=0 + Touche `HAUT` | `sautBooleen`=1, `v_y`=`FORCE_SAUT`. |
| Saut avec déplacement | `sautBooleen`=1 + Touche `DROITE` | `pos.y` et `pos.x` évoluent selon la parabole du saut. |
| Atterrissage sur nuage | `sautBooleen`=1, `pos.y` $\geq$ nuage | `sautBooleen`=0, `v_y`=0, `wait`=20. |
| Transition entre collines | `hors_nuage`=1, nouveau nuage détecté | Appel `replacementNuage`, `id_nuage` change, dino tombe sur nouveau nuage. |

Table 5 – Cas de test valides sur la fonction *deplacement_dino*.

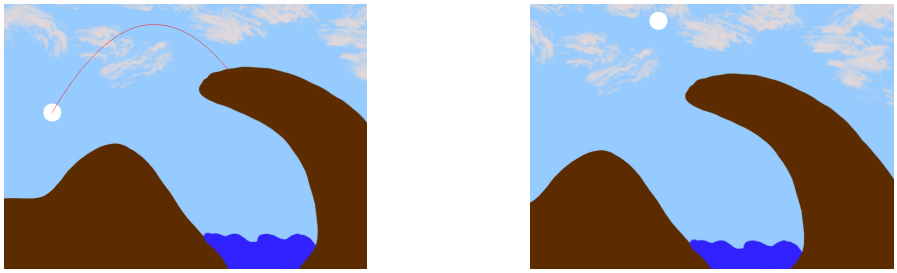
| **Cas de test invalides** | **Données (Condition limite)** | **Résultats attendus** |
|---|---|---|
| Noyade du dinosaure | `matrice`[y][x] == -1 (eau) | Appel de `noyade()`, `etat`=0, `pv`=0 et arrêt des mouvements. |
| Sortie du nuage de points | `hors_nuage`=1 et `replacementNuage` échoue | Passage à l'état `tomber`=2, chute verticale jusqu'à la limite du terrain. |
| Action sur dinosaure mort | `dino->etat` == 0 | La fonction `deplacement_dino` s'arrête immédiatement. |
| Saut impossible (Attente) | `wait` > 0 + Touche `HAUT` | La variable `sautBooleen` reste à 0, aucune impulsion verticale n'est donnée. |
| Blocage par décor | Collision avec `TERRE` lors d'un mouvement | La position `pos.y` est réinitialisée à sa valeur précédente pour empêcher de traverser le sol. |

TABLE 6 – Cas de test invalides sur la fonction *deplacement_ dino*.

## 7.2 Traces de débogage

### 7.2.1 Correction du système de collisions de la bombe

Lors de la création des fonctions chargées de détecter un contact entre la bombe et le terrain, la mer ainsi que les extrémités de la fenêtre, un bug silencieux a été rencontré, necessitant l'usage de l'outil gdb pour identifier le dysfonctionnement. La situation est la suivante :



(a) Trajectoire prévue pour la bombe (b) Arrêt anormal de la bombe

FIGURE 14 – Illustration du parcours incorrect effectué par la bombe

La bombe interrompt sa trajectoire au milieu du ciel avant d'avoir heurté la terre. Il s'agit par conséquent d'identifier une erreur dans une des fonctions liées à la bombe, en particulier dans la fonction *collisionTerrainBombe*. Pour cela, les fichiers impliqués dans l'affichage et le système de collisions sont compilés avec l'option -g :

```
CC = gcc

FLAGS_COMPILATION = -Wall -g `sdl2-config --cflags`
FLAGS_EDITION = `sdl2-config --libs` -lSDL2_image -lSDL2_ttf

PROG = ./bombe

OBJETS = \
    testFonctionsRebonds.o \
    fonctionsRebonds.o \
    fonctionsVerification.o \
    chargerMatrice.o

all: $(PROG)

$(PROG): $(OBJETS)
    $(CC) $(OBJETS) $(FLAGS_EDITION) -o $(PROG)

%.o: %.c
    $(CC) $(FLAGS_COMPILATION) -c $< -o $@
```

FIGURE 15 – Extrait du makefile utilisé pour le débogage des collisions

Les lignes qui suivent constituent la séquence des instructions de débogage mise en oeuvre pour corriger le système de collision. Cette séquence de commande nécessite de provoquer artificiellement l'évènement clavier déclenchant le lancer de la bombe afin d'analyser le contenu des variables lorsque

l'arme est en mouvement puis lors de la collision. Dans le jeu, c'est la touche b du clavier, représentée par la macro SDL_SCANCODE_B dans le module SDL qui amorce le tir de la bombe. Cette macro possédant pour valeur cinq, l'élément d'indice cinq du tableau *etatClavier* qui consigne les touches actionnées par l'utilisateur est par conséquent positionné à un au cours de l'exécution du programme sous gdb.

**Séquence des instructions de débogage :**

```
gdb bombe
# Mise en place des points d'arrêt
# Initialiser l'affichage des variables
(gdb) break testFonctionsRebonds.c :67
# Simuler l'appui sur la touche b du clavier
(gdb) break testFonctionsRebonds.c :92
# Vérifier le contenu des variables au départ du lancement de la
 bombe
(gdb) break testFonctionsRebonds.c :98
# Analyser les valeurs des variables à la suite d'une collision
 avec le terrain
(gdb) break testFonctionsRebonds.c: 117
# Provoquer un arrêt en cas de collision avec le terrain
(gdb) break fonctionsRebonds.c:60 \
if ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE))
# Lancement de l'exécution du programme
(gdb) run
(gdb) display bombeLancee
bombeLancee = 0
(gdb) display bombe.coor
bombe.coor = {x = 325, y = 350}
(gdb) continue
bombeLancee = 0
bombe.coor = {x = 325, y = 350}
(gdb) set etatClavier[5] = 1
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 325, y = 350}
(gdb) continue
Thread 1 "bombe" hit Breakpoint 5, \
rebondTerrainBombe (matriceTerrain=0x7fffffc85290, bombe=0x7fffffc85234) \
at fonctionsRebonds.c:60 \
```

```
return ((matriceTerrain[bombe->coor.x][bombe->coor.y + bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x][bombe->coor.y - bombe->rayon] == TERRE) \
|| (matriceTerrain[bombe->coor.x + bombe->rayon][bombe->coor.y] == TERRE) \
|| (matriceTerrain[bombe->coor.x - bombe->rayon][bombe->coor.y] == TERRE));
(gdb) continue
bombeLancee = 1
bombe.coor = {x = 592, y = 131}
(gdb) next
bombeLancee = 0
bombe.coor = {x = 592, y = 131}
(gdb) quit
```

À l'issue de la séquence de commandes, il s'avère que l'erreur réside dans une inversion de coordonnées dans la fonction *collisionTerrainBombe*, comme l'illustre l'arrêt au point d'arrêt numéro cinq. Cette inversion s'explique par une différence de convention entre la matrice et l'affichage dans la fenêtre avec le module SDL. Tandis que la largeur correspond à l'axe des ordonnées pour la matrice, il s'agit de l'axe des abscisses pour l'affichage et inversement pour la hauteur. La résolution du comportement incohérent de la bombe nécessite donc uniquement d'échanger les coordonnées x et y dans les tests effectués par la fonction.

### 7.2.2 Correction du point d'attache du Grappin

Lors de l'activation du balancier par la touche **ENTREE**, le grappin n'est plus attaché au bon endroit. Normalement il doit ressembler à l'image 16, mais il ressemble à l'image 17. Pour trouver l'erreur, le programme est lancé avec GDB.

**Séquence des instructions de débogage :**

```
gdb ./testGrappin
(gdb)  break balancier
(gdb)  print *pts_rotation
(gdb)  backtrace
(gdb) quit
```

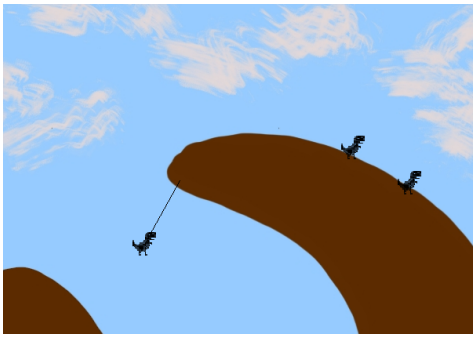


FIGURE 16 – Grappin correctement attaché

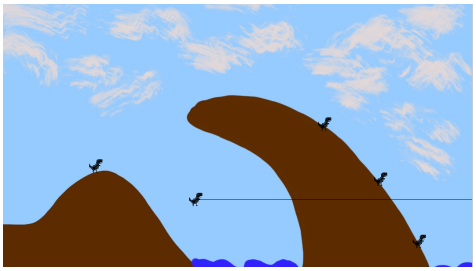


FIGURE 17 – Grappin avec un problème d'attache

Grace à la capture d'écran 18 nous pouvons voir que les valeurs pointées par *pts_rotation* ne rentrent pas dans les limites du terrain, elles sont donc corrompues. Avec backtrace nous comprenons que le problème vient de la fonction *grappin*. En regardant juste au-dessus de la ligne 337, dans *grappin.c*, nous observons que la **file** est détruite. Donc *pts_rotation*, qui doit pointer vers la queue de file, devient un pointeur fou. Pour régler le problème, au lieu de passer l'adresse d'un élément qui va être supprimé, voir image 19,

nous faisons une copie du contenu de cet élément dans une variable locale, voir image 20.

```
Thread 1 "testGrappin" hit Breakpoint 1, balancier (matrice=0x7fffffc85620, dino=0x
7fffffc852d0, zoneAffichage=0x555555a08070, state=0x7ffff7faac6a "", police=0x55555
59156d0, texMap=0x555555987150, texDinos=0x7fffffc853a0, texObjets=0x7fffffc853d0,
nomsObjets=0x7fffffc85410, equipe1=0x7fffffc85300, equipe2=0x7fffffc85330, pts_rota
tion=0x5555559308d0, nb_pts=0x7fffffc85284, nuage=0x7fffffc852c8, nb_nuage=2, nomNu
age=0x7fffffc85360) at ../src/grappin.c:79
79          if (pts_rotation == NULL || (*dino) == NULL) return;
(gdb) print *pts_rotation
$1 = {x = 12997072, y = 21840}
(gdb) backtrace
#0  balancier (matrice=0x7fffffc85620, dino=0x7fffffc852d0,
    zoneAffichage=0x555555a08070, state=0x7ffff7faac6a "", police=0x5555559156d0,
    texMap=0x555555987150, texDinos=0x7fffffc853a0, texObjets=0x7fffffc853d0,
    nomsObjets=0x7fffffc85410, equipe1=0x7fffffc85300, equipe2=0x7fffffc85330,
    pts_rotation=0x5555559308d0, nb_pts=0x7fffffc85284, nuage=0x7fffffc852c8,
    nb_nuage=2, nomNuage=0x7fffffc85360) at ../src/grappin.c:79
#1  0x0000555555558983 in grappin (matrice=0x7fffffc85620,
    zoneAffichage=0x555555a08070, dino=0x7fffffc852d0, state=0x7ffff7faac6a "",
    texMap=0x555555987150, police=0x5555559156d0, texDinos=0x7fffffc853a0,
    texObjets=0x7fffffc853d0, nomsObjets=0x7fffffc85410, equipe1=0x7fffffc85300,
    equipe2=0x7fffffc85330, nb_pts=0x7fffffc85284, nuage=0x7fffffc852c8,
    nb_nuage=2, nomNuage=0x7fffffc85360) at ../src/grappin.c:335
#2  0x00005555555572e1 in main (argc=1, argv=0x7fffffffe208)
    at ../test/testGrappin.c:138
(gdb)
```

FIGURE 18 – Capture d'écran du débogage de *test_grappin*

```
    else if(etat_fini == 3) {
        t_coordonnee *pivot_fixe= lireQueue();
        detruireFile(); // On peut maintenant vider la file sans risque

        balancier(matrice, dino, zoneAffichage, state, police, texMap,
            texDinos,texObjets, nomsObjets, equipe1, equipe2, pivot_fixe,
            nb_pts,nuage,nb_nuage,nomNuage);
        if((*dino)->etat==0){
            supprimerDinoJoueur(equipe1, equipe2, (*dino)->d);
            *dino = NULL;
            return 0;
        }
    }
}
```

FIGURE 19 – Image du code erroné de *grappin.c*

```c
}
else if(etat_fini == 3) {
    t_coordonnee *res = lireQueue();
    if (res != NULL) {
        t_coordonnee pivot_fixe = *res; // COPIE DES VALEURS
        detruireFile(); // On peut maintenant vider la file sans risque

        balancier(matrice, dino, zoneAffichage, state, police, texMap,
            texDinos,texObjets, nomsObjets, equipe1, equipe2, &pivot_fixe,
            nb_pts,nuage,nb_nuage,nomNuage);
        if((*dino)->etat==0){
            supprimerDinoJoueur(equipe1, equipe2, (*dino)->d);
            *dino = NULL;
            return 0;
        }
    }
```

FIGURE 20 – Code corrigé de *grappin.c*

### 7.2.3 Correction des gestions de zones pour le placement des dinosaures

Lors du développement du module de placement des dinosaures, une erreur logique a été identifiée dans la fonction *generer_ catalogue_ depuis_ nuage*. Alors que le nuage de points contient des coordonnées valides, le programme ne parvient pas à allouer les dix zones de spawn requises pour chaque équipe, déclenchant systématiquement le message d'alerte "Attention : Zones de spawn incomplètes !" de sécurité après 5000 itérations.

Ainsi puisque le programme n'affiche aucune erreur mais que les dinosaures n'apparaissent pas, on utilise gdb pour observer l'exécution 'en direct'. En utilisant la commande display dino, on s'aperçoit que même si dino.x est inférieur à LARGEUR_TERRAIN/3 (ce qui est bien dans le tiers gauche du terrain), le programme saute l'instruction d'ajout. En regardant la ligne de code source affichée par gdb, on réalise que le test porte sur dino.y. Comme dino.y représente la hauteur et non la largeur, la condition n'est jamais remplie si le point est trop haut ou trop bas sur la carte.

**Séquence des instructions de débogage :**

```
gdb ./main

# Mise en place des points d'arrêt
(gdb) break ../src/gestion_zones.c:26
Breakpoint 1 at 0xae30: file ../src/gestion_zones.c, line 26.

# Lancement de l'exécution du programme
(gdb) run
Partie Lancée !

Thread 1 "main" hit Breakpoint 1, generer_catalogue_depuis_nuage (
    nuage=0x555559054ae0, nb_points=551, catalogue=0x7fffffffdce0,
    nb_E1=0x7fffffffda64, nb_E2=0x7fffffffda68, id_n=0) at ../src/gestion_zones.c:26
26          while (points_ajoutes_ce_tour < 10 && securite < 5000) {
(gdb) step
27 trop_proche = 0;
(gdb) step
28 securite++;
(gdb) step
30 r = rand() % nb_points;
(gdb) next
generer_catalogue_depuis_nuage (nuage=0x555559054ae0, nb_points=551,
    catalogue=0x7fffffffdce0, nb_E1=0x7fffffffda64, nb_E2=0x7fffffffda68, id_n=0)
    at ../src/gestion_zones.c:31
```

```
31 t_coordonnee dino = nuage[r];
(gdb) display dino
1: dino = {x = -809228544, y = 103994526}
(gdb) step
34 for(i=0; i < *nb_E1; i++) {
1: dino = {x = 315, y = 488}
(gdb) step
39 for(i=0; i < *nb_E2; i++) {
1: dino = {x = 315, y = 488}
(gdb) step
45 if(!trop_proche) {
1: dino = {x = 315, y = 488}
(gdb) step
47 if ((dino.y < (LARGEUR_TERRAIN / 3)) && (*nb_E1 < 10)) {
1: dino = {x = 315, y = 488}
(gdb) step
56 else if ((dino.y > (2 * LARGEUR_TERRAIN / 3)) && (*nb_E2 < 10)) {
1: dino = {x = 315, y = 488}
(gdb) step
26 while (points_ajoutes_ce_tour < 10 && securite < 5000) {
(gdb) quit
```